15 Stream processing and incremental I/O

We said in the introduction to part 4 that functional programming is a *complete* paradigm. Every imaginable program can be expressed functionally, including programs that interact with the external world. But it would be disappointing if the `IO` type were the only way to construct such programs. `IO` and `ST` work by simply embedding an imperative programming language into the purely functional subset of Scala. While programming within the `IO` monad, we have to reason about our programs much like we would in ordinary imperative programming.

We can do better. In this chapter, we'll show how to recover the high-level compositional style developed in parts 1–3 of this book, even for programs that interact with the outside world. The design space in this area is enormous, and our goal here is not to explore it completely, but just to convey ideas and give a sense of what's possible.

15.1 Problems with imperative I/O: an example

We'll start by considering a simple concrete usage scenario that we'll use to highlight some of the problems with imperative I/O embedded in the `IO` monad. Our first easy challenge in this chapter is to write a program that checks whether the number of lines in a file is greater than 40,000.

This is a deliberately simple task that illustrates the essence of the problem that our library is intended to solve. We could certainly accomplish this task with ordinary imperative code, inside the `IO` monad. Let's look at that first.

Listing 15.1 Counting line numbers in imperative style

```
def linesGt40k(filename: String): IO[Boolean] = IO {
  val src = io.Source.fromFile(filename)
  try {
    var count = 0
    val lines: Iterator[String] = src.getLines
    while (count <= 40000 && lines.hasNext) {
      lines.next
      count += 1
    }
    count > 40000
  }
  finally src.close
}
```

`scala.io.Source` has convenience functions for reading from external sources like files.

Obtain a stateful `Iterator` from the `Source`.

Has the side effect of advancing to the next element.

We can then *run* this `IO` action with `unsafePerformIO(linesGt40k("lines.txt"))`, where `unsafePerformIO` is a side-effecting method that takes `IO[A]`, returning `A` and actually performing the desired effects (see section 13.6.1).

Although this code uses low-level primitives like a `while` loop, a mutable `Iterator`, and a `var`, there are some *good* things about it. First, it's *incremental*—the entire file isn't loaded into memory up front. Instead, lines are fetched from the file only when needed. If we didn't buffer the input, we could keep as little as a single line of the file in memory at a time. It also terminates early, as soon as the answer is known.

There are some bad things about this code, too. For one, we have to remember to close the file when we're done. This might seem obvious, but if we forget to do this, or (more commonly) if we close the file outside of a `finally` block and an exception occurs first, the file will remain open.[1] This is called a *resource leak*. A file handle is an example of a scarce resource—the operating system can only have a limited number of files open at any given time. If this task were part of a larger program—say we were scanning an entire directory recursively, building up a list of all files with more than 40,000 lines—our larger program could easily fail because too many files were left open.

We want to write programs that are *resource-safe*—they should close file handles as soon as they're no longer needed (whether because of normal termination or an exception), and they shouldn't attempt to read from a closed file. Likewise for other resources like network sockets, database connections, and so on. Using `IO` directly can be problematic because it means our programs are entirely responsible for ensuring their own resource safety, and we get no help from the compiler in making sure that they do this. It would be nice if our library would ensure that programs are resource-safe by construction.

But even aside from the problems with resource safety, there's something unsatisfying about this code. It entangles the high-level algorithm with low-level concerns about iteration and file access. Of course we have to obtain the elements from some

[1] The JVM will actually close an `InputStream` (which is what backs a `scala.io.Source`) when it's garbage collected, but there's no way to guarantee this will occur in a timely manner, or at all! This is especially true in generational garbage collectors that perform "full" collections infrequently.

resource, handle any errors that occur, and close the resource when we're done, but our program isn't *about* any of those things. It's about counting elements and returning a value as soon as we hit 40,000. And that happens *between* all of those I/O actions. Intertwining the algorithm and the I/O concerns is not just ugly—it's a barrier to composition, and our code will be difficult to extend later. To see this, consider a few variations of the original scenario:

- Check whether the number of *nonempty* lines in the file exceeds 40,000.
- Find a line index before 40,000 where the first letters of consecutive lines spell out `"abracadabra"`.

For the first case, we could imagine passing a `String => Boolean` into our `linesGt40k` function. But for the second case, we'd need to modify our loop to keep track of some further state. Besides being uglier, the resulting code will likely be tricky to get right. In general, writing efficient code in the `IO` monad generally means writing monolithic loops, and monolithic loops are not composable.

Let's compare this to the case where we have a `Stream[String]` for the lines being analyzed:

```
lines.zipWithIndex.exists(_._2 + 1 >= 40000)
```

Much nicer! With a `Stream`, we get to assemble our program from preexisting combinators, `zipWithIndex` and `exists`. If we want to consider only nonempty lines, we can easily use `filter`:

```
lines.filter(!_.trim.isEmpty).zipWithIndex.exists(_._2 + 1 >= 40000)
```

And for the second scenario, we can use the `indexOfSlice` function defined on `Stream`,[2] in conjunction with `take` (to terminate the search after 40,000 lines) and `map` (to pull out the first character of each line):

```
lines.filter(!_.trim.isEmpty).
      take(40000).
      map(_.head).
      indexOfSlice("abracadabra".toList)
```

We want to write something like the preceding when reading from an actual file. The problem is that we don't have a `Stream[String]`; we have a file from which we can read. We could cheat by writing a function `lines` that returns an `IO[Stream[String]]`:

```
def lines(filename: String): IO[Stream[String]] = IO {
  val src = io.Source.fromFile(filename)
  src.getLines.toStream append { src.close; Stream.empty }
}
```

This is called *lazy I/O*. We're cheating because the `Stream[String]` inside the `IO` monad isn't actually a pure value. As elements of the stream are forced, it'll execute

[2] If the argument to `indexOfSlice` doesn't exist as a subsequence of the input, it returns `-1`. See the API docs for details, or experiment with this function in the REPL.